Process

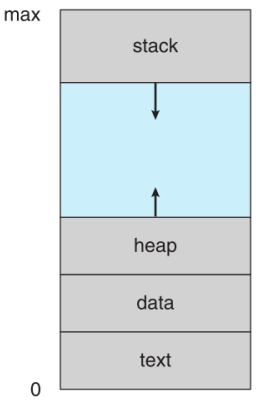


Figure 3.1 Layout of a process in memory.

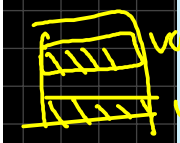
t1 fun{ val1 = ; }

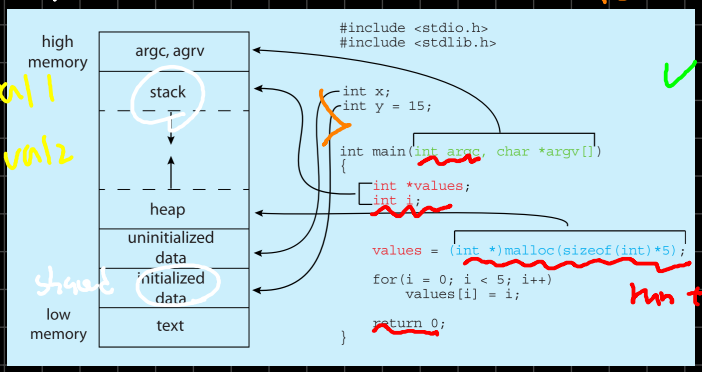
t2 fun{ val2 = ; }

CoW

int func( ) { static int val1 = (int*)malloc; }

val2 = heap





Stack : temporary data in function
(e.g. function parameters, return address, local variable)

heap : dynamically allocated during run time

data : global variable

```c
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;            // Data segment
static int static_var = 20;     // Data segment

int function(int para) // Stack
{
    int local_var = 30;         // Stack

    return 0; // Stack
}

int main()
{
    int local_var = 40;         // Stack

    int *heap_var = malloc(sizeof(int)); // Heap
    *heap_var = 50;

    free(heap_var);

    return 0;
}
```

Thread

race condition

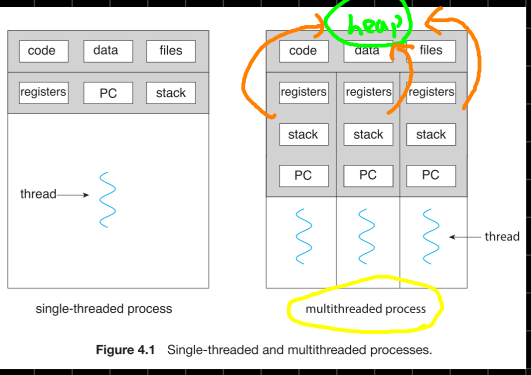


Figure 4.1 Single-threaded and multithreaded processes.

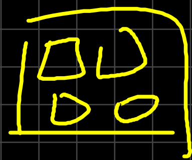
Shared : code, data, heap, files

private : registers, stack, PC

Thread

Which item(s) is(are) shared by threads of a multi-threaded process?
(a) static local variables
(b) program text/executable binaries
(c) register values of the CPU
(d) heap memory

交大99



(c) reg.s } private   PC, stack
(next address) (stack pointer)

✓ (a) static local vars → global var. (data)
✓ (b) code
✓ (d) heap
} shared

5. (5 points) Which of the following components of program stat are shared across threads in a multithreaded process? (A). Register values (B).Heap memory (C).Global variables (D).Stack memory

中央 102

✓ (B) heap
✓ (C) global variables (data)
} shared

thread model

kernel \ user	1	multi
1	s / s	sss / s
multi	s / sss	sss / sss

Linux

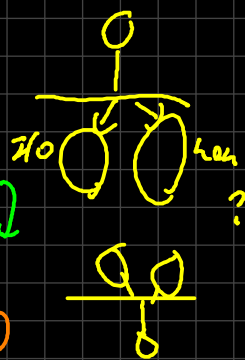
CPU core ↔ thread
CPU 基本執行單位

× 1 core ← 多 thread

13. Which of the following statements are correct?
(a) A process with multiple user threads and one kernel thread (many-to-one) cannot run in parallel on multiprocessors.
(b) Threads belonging to a process share one program counter.
(c) Operating system schedule kernel threads and user threads as well.
(d) Most operating systems, such as Solaris and Windows, boost the thread priority when it runs out of time quantum.
(e) Most operating systems, such as Solaris and Windows, boost the thread priority when it returns from an I/O request.

core

交大100



sys.call — user mode / kernel mode thread

OS = kernel

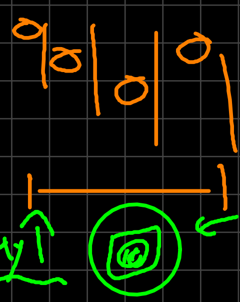
✓ (a) user ∵ 只有1個 kernel thread
kernel thread, CPU core
∴ { parallel × (多CPU, CPU 同時執行)
concurrent (1 CPU run multi-thread using time slicing)

× (b) program counter ⇒ private

× (c) OS schedule kernel threads

× (d) run out time quantum → CPU bound

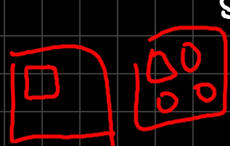
✓ (e) I/O request → I/O bound
increase priority



priority ↑

ch1 multi-program, multi-task
ch3 process
ch4 thread — multi-thread — thread model

32 core 64 thread



user/kernel SW ↓ HW thread

4. (3 points)

When a process is created using the classical fork() system call, which of the following are NOT inherited by the child process from the parent process?

(A) process address space

(B) process ID

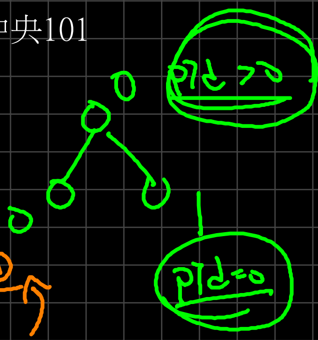
(C) user ID

(D) process group ID → linux

(E) none of the above

中央101

child pid = 0



user ID

user, parent

16. Which threading model is supported by the Linux operating system?

(a) one-to-many

(b) many-to-one

(c) one-to-one

(d) many-to-many

(e) none of the above

中央 102

linux : one-to-one

user

kernel

(a) [2 points] The more threading, the more performance.

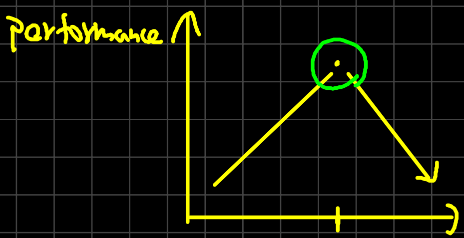
台大 109

false, (1) # CPU (core) 有限

(2) thread 太多，需要很多 context switch (overhead)

1000

8 core



Appendix.

1 CPU → 多 thread

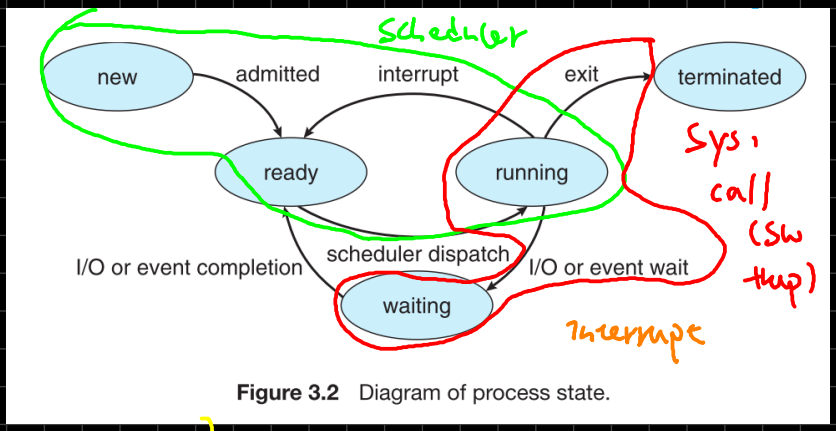


Figure 3.2 Diagram of process state.

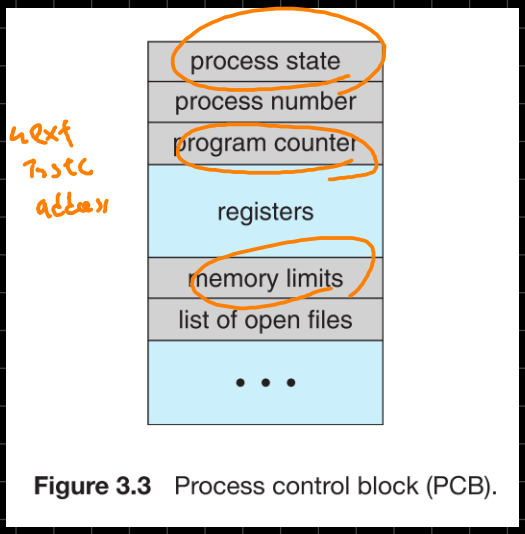


Figure 3.3 Process control block (PCB).

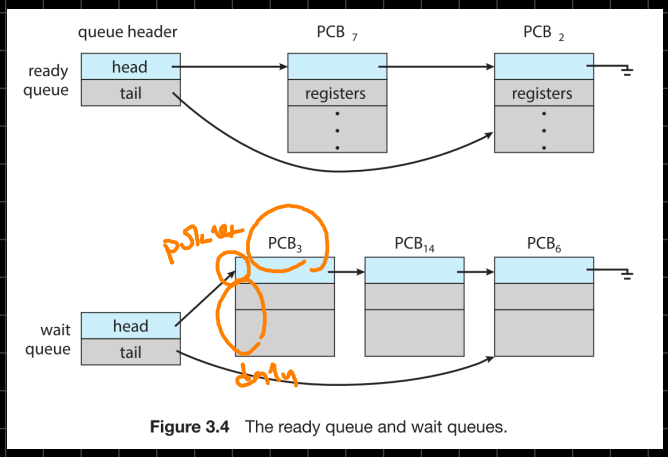


Figure 3.4 The ready queue and wait queues.

Context Switch

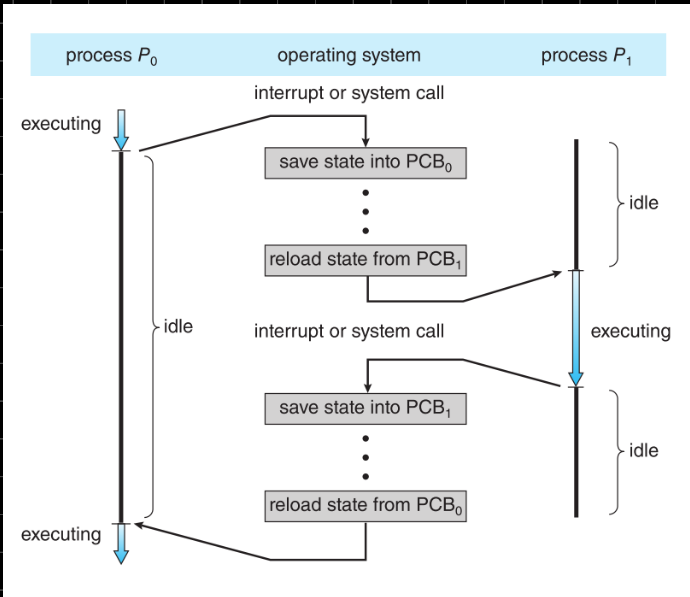


Figure 3.6 Diagram showing context switch from process to process.

OS

(1) suspend, then resume it

(2) purely overhead

Interrupt

SW ?

x

(1) change CPU core from current task to run a kernel routine

PCB

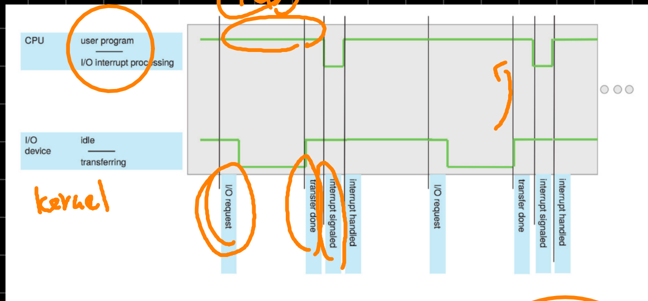


Figure 1.3 Interrupt timeline for a single program doing output.

sys stall,

kernel

CPU (HW)

event listening

event

while {

}